

Stoquify Agent Runtime Roadmap

Complete Analysis, Platform Impact, Market Impact, Value Proposition, Risk Review, and Recommendations

Date: 2026-07-22

Prepared from: STOQUIFY_AGENT_RUNTIME_ROADMAP_REFINED_PROMPT_2026-07-21.pdf and STOQUIFY_AGENT_RUNTIME_PROPOSAL_REPORT_2026-07-21.md

Scope: strategic, market, product, architecture, security, compliance, data, UX, go-to-market, implementation, and success-measurement analysis.

Core conclusion: Stoquify should pursue the proposal, but as a Stoquify-native controlled operating-intelligence runtime built on protected services, evidence, RBAC, module entitlement, redaction, action drafts, and human approval. The winning product is not generic AI chat. The winning product is trusted daily action with evidence for African and OHADA SMBs.

1. Executive Summary

The proposal is strategically sound and technically aligned with Stoquify's existing architecture. Its biggest promise is to turn Stoquify from a system that records and displays operational data into a system that interprets daily business reality, ranks what matters, explains evidence, drafts safe action, and routes work to the right human role.

The roadmap should be pursued, but with strict boundaries. Agents should read, explain, recommend, draft, and route. They should not directly post ledgers, file statutory declarations, approve payroll, certify close packs, change permissions, adjust cash, write off stock, approve stock counts, or write directly to Prisma.

The highest-leverage starting point is a read-only Command Agent using existing snapshots, business signals, action queues, proof trails, payment reconciliation read models, and inventory cash truth. Cash/Reconciliation and Inventory/Replenishment agents should follow in read-and-draft mode. Payroll, statutory, close certification, ledger posting, and permission-changing actions should remain later-stage and human-controlled.

Overall assessment

Dimension	Assessment	Implication
Strategic fit	High	Moves Stoquify toward a defensible operating command layer, not a generic AI add-on.
Market upside	High	Creates a stronger category claim for African and OHADA SMBs: evidence-backed daily operating intelligence.
Technical feasibility	Medium-high	Existing services, RBAC, snapshots, action queue, proof trails, and redaction lower implementation risk.
Execution complexity	Medium-high	Requires schema, tool registry, observability, cost control, evaluations, and release gates.
Security/compliance risk	High if uncontrolled	Must enforce no-direct-write, redaction-before-model, approval, fresh auth, audit, and evidence requirements.
Recommended start	Narrow	Read-only Command Agent and internal runtime before any execution capability.

2. What The Proposal Really Means

The PDF is a refined execution prompt asking for a full roadmap. The proposal report contains the substantive recommendation: build one shared Stoquify-native agent runtime that turns existing business truth into evidence-backed decisions and controlled action drafts.

The proposal is not asking Stoquify to become an AI chatbot product. It is asking Stoquify to make its operational truth more usable. The system should answer practical questions such as: what changed today, where is cash blocked, which stock risks need attention, which payroll or compliance item is not ready, who can resolve it, what evidence supports the recommendation, and what controlled workflow should happen next.

- One shared runtime, not one agent framework per module.
- Service-owned tools only, with no raw Prisma tools exposed to agents.
- Evidence and redaction before any model output.
- Recommendations and action drafts before execution.
- Human approval, fresh auth, maker-checker, and idempotency for risky actions.
- Role-aware UI surfaces embedded into command centers, evidence drawers, action queues, and module workbenches.

3. Strategic North Star

The end-state product should be: Stoquify is the trusted command system for SMB operations, finance, inventory, payroll, compliance, close assurance, and daily decisions. It should not simply display dashboards. It should tell each user what matters, why it matters, what proof exists, what risk remains, and what safe workflow should resolve it.

This north star is defensible because Stoquify already has the hard-to-copy ingredients: tenant-scoped operational data, POS, inventory, payments, reconciliation, payroll, close, compliance, RBAC, module entitlement, evidence grades, proof trails, redaction, workflow assurance, and release gates. The agent runtime makes those assets visible and habit-forming.

Strategic transformation

Current product perception	Target perception after roadmap
Business management SaaS	Daily operating intelligence system
Dashboards and records	Evidence-backed priorities and controlled action
Module-by-module workflows	Cross-module command layer across cash, stock, payroll, close, and compliance
Users search for blockers	Stoquify surfaces blockers and routes owners
AI as a feature	AI as governed runtime embedded into Stoquify's trust architecture

4. Market-Level Impact

The roadmap creates a category shift. Stoquify would no longer be judged only against POS, inventory, accounting, payroll, ERP, or dashboard tools. It can occupy a stronger position: an evidence-backed command system for African and OHADA SMBs.

This matters because most SMB tools are passive systems of record. They record transactions and expose reports. Stoquify can become a system of operating judgment. It can tell users which risks require attention now, how those risks connect across cash, stock, payroll, close, and compliance, and what controlled action should happen next.

Competitive positioning

Competitor category	Typical value	Stoquify agent-runtime advantage
POS	Record sales and payments	Explain cash truth, drawer variances, payment capture gaps, duplicate references, and reconciliation readiness.
Inventory	Show stock on hand	Explain stockout risk, negative stock, dead stock cash exposure, purchasing delays, and stock-to-cash effects.
Accounting	Show ledgers and reports	Explain close blockers, source evidence gaps, audit trails, and required owner actions.
Payroll	Calculate payslips	Explain payroll readiness, missing contracts, attendance drift, declarations, payment evidence, and redacted person-data states.
Generic AI chatbot	Answer broad questions	Provide tenant-safe, permission-filtered, evidence-backed operational guidance inside controlled workflows.
ERP	Centralize transactions	Reduce decision friction through role-aware briefs, action drafts, exception queues, and proof-backed recommendations.

Recommended market claim

Recommended positioning: Stoquify is the evidence-backed operating command system for African and OHADA SMBs, connecting cash, stock, payroll, compliance, close, and daily work into trusted decisions and controlled action.

Avoid positioning it as 'AI for SMBs'. That claim is too generic and easily copied. The differentiated claim is trust, regional business context, evidence, and workflow execution.

5. Value Proposition Impact

The roadmap upgrades the value proposition from connected business modules to role-specific operating intelligence. Users would not open Stoquify only to enter records or inspect dashboards. They would open it daily to understand what needs attention and what action should happen next.

Persona	Before	After agent runtime
Owner	Checks dashboards and reports	Gets a daily command brief: money blocked, risks, approvals, unresolved work, and evidence.
Manager	Tracks module-level tasks	Sees stale, blocked, assigned, and high-risk operational work across modules.
Accountant	Investigates reconciliation and close manually	Receives proof-backed blocker explanations, close readiness gaps, and action drafts.
Finance officer	Reviews payment exceptions	Gets ranked cash and reconciliation triage with exposure, evidence quality, and draft next steps.
Stockkeeper	Monitors stock lists	Gets stock risk, dead stock exposure, stockout warnings, count-review drafts, and replenishment suggestions.
HR/payroll lead	Checks payroll readiness surfaces	Gets payroll blockers, declaration readiness, payment evidence gaps, redacted states, and close exposure.
Auditor	Requests proof manually	Gets evidence-linked narratives with source hashes, freshness, redactions, and audit trail references.

Value proposition formula

A practical product message could be: Stoquify does not just manage your business records. It tells your team what matters today, proves why, and helps route the safe next action.

6. Product And UX Impact

The agent experience should feel like Stoquify became more operationally intelligent, not like a floating chatbot was added. The proposal correctly recommends embedded command surfaces rather than generic AI hero pages or chat everywhere.

- Daily brief: role-aware summary in Daily Digest and Owner War Room.
- Command panel: surface today's operating truth, blockers, priorities, and evidence.
- Module assistant panel: explain this, why blocked, what next, show evidence.
- Evidence drawer: extend proof trails with agent evidence links, source hashes, redaction notices, and freshness.
- Action draft cards: visible summary, affected records, risk level, required permission, approval state, evidence grade, and service path.
- Approval timeline: requested, reviewed, approved, rejected, executed, failed, or expired.
- Exception queue: integrated with Manager Action Center and Assurance Control Tower.

The main UX risk is chatbot fatigue. If the user must ask the right question, the product will feel generic. Stoquify should proactively surface the most relevant daily operating questions and make agent output scannable, evidence-backed, and actionable.

7. Architecture Impact

The proposed architecture is directionally correct: one runtime domain, service-owned tools, evidence binding, action drafts, approvals, observability, and policy incidents. The runtime should not own business truth. It should own execution metadata, evidence linkage, approval state, and safe presentation.

Recommended runtime modules

Module	Purpose
agent-context.service.ts	Resolve tenant, actor, role, permissions, locale, currency, module entitlements, route context, and available read models.
agent-tool-registry.service.ts	Expose only service-owned tools with schemas, permissions, module scope, risk level, and approval policy.

Module	Purpose
agent-runner.service.ts	Coordinate model calls, tool loops, prompt versions, step logging, safe errors, and runtime status.
agent-evidence.service.ts	Bind every answer and recommendation to snapshots, proof trails, source hashes, evidence grades, blockers, freshness, and redactions.
agent-redaction.service.ts	Apply redaction before prompt construction and before final response display.
agent-action-draft.service.ts	Create draft payloads for existing protected service actions with idempotency keys and risk metadata.
agent-approval.service.ts	Handle fresh auth, maker-checker, signed input hash, approval expiry, rejection, and replay checks.
agent-observability.service.ts	Track traces, latency, cost, model policy, feedback, unsafe attempts, and policy incidents.

Data model implications

The proposed models are justified, but should be introduced gradually. Do not add every table before the first read-only pilot proves usage. The minimum set for phase one is AgentDefinition, AgentToolDefinition, AgentRun, AgentStep, AgentEvidenceLink, AgentFeedback, and AgentCostLedger. AgentActionDraft and AgentApproval can come in phase two when draft workflows begin. AgentPolicyIncident should exist early if prohibited tools or unsafe outputs are being tested.

8. Security, Privacy, And Compliance Analysis

The security model is the make-or-break issue. The proposal is valuable because it keeps agents inside Stoquify's protected service model. It becomes dangerous if agents are allowed to bypass RBAC, module entitlement, redaction, audit, fresh auth, maker-checker, or service boundaries.

Non-negotiable prohibitions

- No direct Prisma writes by agents.
- No direct ledger posting.
- No statutory filing by agents.
- No payroll approval or payroll payment release by agents.
- No permission, role, or entitlement changes by agents.
- No close certification or close waiver approval by agents.
- No cash adjustment, stock write-off, stock count approval, or purchase order approval by agents.
- No sensitive exports without permission checks and redaction.

Control model

The controlled path should be: permission-filtered context, redacted model input, evidence-backed explanation, action draft, server-side validation, approval/fresh auth if needed, service-owned execution, audit event, evidence link, and feedback capture.

For OHADA/SYSCOHADA and payroll/compliance content, the agent must show provenance and limitations. It should not present non-reviewed statutory interpretation as legal truth.

9. Technology Adoption Review

The proposal's technology recommendations are reasonable, with one important discipline: adopt fewer external dependencies at first. Stoquify should avoid combining multiple overlapping agent gateways or durable execution systems before traffic and workflows justify them.

Technology	Recommendation	Reason
AI SDK or TypeScript-native equivalent	Adopt first	Fits Next.js/TypeScript/Zod and supports typed tools, streaming, runtime context, approvals, and telemetry.
MCP-compatible tool shapes	Design early, expose later	Good boundary for tools, but governance must remain Stoquify-owned.
Inngest or Trigger.dev	Evaluate one	Useful for approval waits, retries, scheduled briefs, and durable tasks. Do not adopt both.
Langfuse	Adopt after pilot traffic	Best for traces, evaluations, feedback loops, and trust monitoring if redacted.
Helicone	Consider for gateway/cost control	Useful for routing and cost visibility, but overlaps with Langfuse.
LiteLLM	Defer	Useful later for multi-provider routing and budgets, not required for MVP.

10. Go-To-Market And Commercial Impact

The roadmap can support a stronger GTM story because it converts Stoquify from a back-office system into a daily decision habit. The commercial narrative should focus on operational control, cash protection, stock discipline, close confidence, payroll readiness, and audit trust.

Recommended packaging

- Core platform: dashboards, modules, records, workflows, and baseline reports.
- Command Intelligence add-on: Command Agent, role daily briefs, evidence-backed blocker explanations, action queue intelligence, and agent run history.
- Finance Trust add-on: Cash/Reconciliation Agent, suspense triage, duplicate reference analysis, payment evidence requests, and reconciliation drafts.
- Inventory Intelligence add-on: stock risk briefs, replenishment drafts, transfer review, dead stock exposure, and count variance suggestions.
- Assurance tier: close/compliance/payroll readiness agents, audit narratives, report trust export, policy incident monitoring, and advanced approval workflows.

Pricing implications

Agent features should not be priced only by token usage. The pricing logic should combine module entitlement, role surfaces, agent run volume, and value. Cash and inventory agents can be priced around operational risk reduction and daily command value. Higher-risk compliance, close, and payroll agents can sit in premium assurance packages.

11. Implementation Roadmap

The roadmap should remain phased and conservative. The first milestone should prove trusted context and evidence-grounded read-only recommendations before any controlled action execution is introduced.

Phase	Goal	Acceptance criteria
0. Design freeze	Define runtime contracts, prohibited tools, model data policy, and first tool set.	Architecture review, security review, no-direct-write policy, source docs, and release gates defined.
1. Shared runtime	Add schema, services/agents, tool registry, run logger, evidence binder, and redaction wrapper.	Tenant isolation, permission filtering, redaction, stale snapshot, and no-write tests pass.
2. Command Agent	Read-only role daily brief from snapshots, signals, action queue, and proof trails.	Internal users can open briefs; every item has evidence, freshness, and redaction state.

Phase	Goal	Acceptance criteria
3. Cash/Reconciliation draft	Read-and-draft triage for cash variance, duplicates, suspense, and pending payments.	No postings; suggestions are drafts with risk and permission metadata.
4. Inventory draft	Read-and-draft stock risk, reorder suggestions, transfer review, and count-review tasks.	No adjustments or write-offs; drafts route through existing workflows.
5. Controlled low-risk actions	Allow selected protected service actions after approval and idempotency checks.	Signed input hash, fresh auth, maker-checker, service action execution, and audit evidence.
6. Trust monitoring	Add cost, trace, feedback, evaluations, policy incidents, and stale-answer tracking.	Dashboards show acceptance, unsafe attempts, cost spikes, stale rates, and failure patterns.
7. Expansion	Purchasing/AP, close/compliance, payroll readiness, and customer success agents.	Each new agent passes domain-specific safety and evidence gates.

12. Metrics And Success Criteria

Success should be measured by usage, operational improvement, trust, and safety. Do not use model activity alone as the success metric.

- Product usage: percentage of pilot users opening Command Agent at least four days per week.
- Recommendation value: accepted or routed percentage for high-confidence recommendations.
- Time savings: user-reported reduction in time to understand daily priorities.
- Cash impact: reduction in open suspense amount, duplicate reference resolution time, and reconciliation exception age.
- Inventory impact: reduction in negative stock blockers, zero-stock incidents, and dead stock cash exposure.
- Trust: percentage of recommendations with evidence grade, source module, freshness, and source hash.
- Safety: zero direct Prisma write tools; zero redaction bypasses; logged and reviewable policy incidents.
- Cost: model cost per tenant/module/user journey, plus budget alerts and route-by-risk policy.

13. Risk Matrix

Risk	Severity	Mitigation
Parallel business logic	High	Tools must wrap existing services and read models; no raw data mutation tools.
Prompt data leakage	High	Redaction before prompt construction, safe summaries in traces, restricted observability payloads.
Unsafe automation	Critical	Read/draft first; approval, fresh auth, maker-checker, idempotency, and service-owned execution only.
Non-authoritative compliance advice	High	Country-pack provenance, reviewed rule metadata, citations, and non-authoritative states.
Generic chatbot fatigue	Medium	Embedded command panels, evidence drawers, exception queues, and role briefs.
Model cost growth	Medium-high	Snapshot caching, summarization, budgets, small-model routing, and cost ledger.
Stale or wrong recommendations	High	Freshness gates, source hashes, stale labels, feedback, and evaluations.
Durable workflow sprawl	Medium	Choose one durable execution layer, version workflows, require idempotency, centralize monitoring.

14. Final Recommendations

- Proceed with the agent runtime roadmap, but explicitly position it as operating intelligence with evidence, not generic AI chat.
- Begin with a small read-only runtime and Command Agent using existing service-owned read models.
- Prioritize Cash/Reconciliation and Inventory/Replenishment before payroll/statutory/close automation because they are frequent, concrete, and commercially clear.
- Introduce AgentActionDraft and AgentApproval only after users validate read-and-draft workflows.
- Keep all dangerous actions human-owned and service-owned indefinitely unless a specific domain passes explicit risk, evidence, approval, and legal review gates.
- Adopt AI SDK or a TypeScript-native equivalent first; design MCP-compatible tool contracts early; defer multi-gateway complexity until usage requires it.
- Build a trust dashboard early: costs, traces, stale answers, unsafe attempts, policy incidents, acceptance rate, and feedback.
- Use the agent runtime to strengthen pricing and packaging around Command Intelligence, Finance Trust, Inventory Intelligence, and Assurance tiers.

The best strategic framing is: Stoquify helps African and OHADA SMBs run the business from evidence, not guesswork. The agent runtime should make that promise visible every day.